

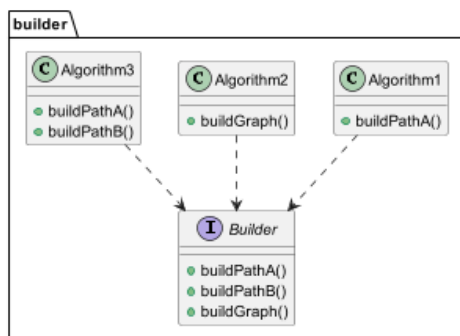
STEP #1: AUDIT YOUR CODE

After exploring topics including developer responsibility, testing, Gang of Four (GoF) patterns and Anti-patterns from weeks 5 to 12, I gained insights into how these concepts facilitate the decomposition of business logic and enhance code structure.

Reflecting on my Maze Runner project from A1, I believe it demonstrated a good object-oriented design. I implemented a basic interface in the /algorithms directory, ensuring consistency input and output across different path-based algorithms. Additionally, the /utils, /parameter directories hold the SOLID principles, promoting manageable code. However, with a deeper understanding of design patterns, I recognize opportunities for improvement. Incorporating more patterns (e.g. Factory, Builder, Adaptor) could significantly improve the current structure, especially by distinguishing between graph-based and non-graph-based algorithms.

The substantial technical debt in my initial delivery is primarily associated with the Maze.java file, which assumed excessive responsibilities, similar to a "god class." This file was tasked with managing the maze's structure, finding paths, and tracking positions, which deviated from the SOLID design principles. Moreover, its design was more aligned with path-based logic rather than a generic algorithm, making it challenging to adapt and extend for more graph-based algorithms. This god class, or anti-pattern led to high coupling and reduced flexibility.

For A3, I would apply a builder pattern for all algorithms as priority optimization.



STEP #2: A GRAPH PROBLEM

How can one formalize this maze exploration problem as a graph one ?

The maze exploration problem can be formalized as a graph problem by representing the maze as a graph where:

Nodes: Each coordinate (x,y) in the maze, which can be a passable area or a blocked area, is represented as a node in the graph.

Edges (Optional for BFS and DFS): An edge connects two nodes if there is a direct passage between the corresponding coordinates in the maze. In a relatively small maze, edges can be undirected, in more giant mazes with one-way passages, directed edges can be used.

Weights (For Dijkstra): If all passages are equally traversable, the graph can be unweighted with all edges having the same cost.

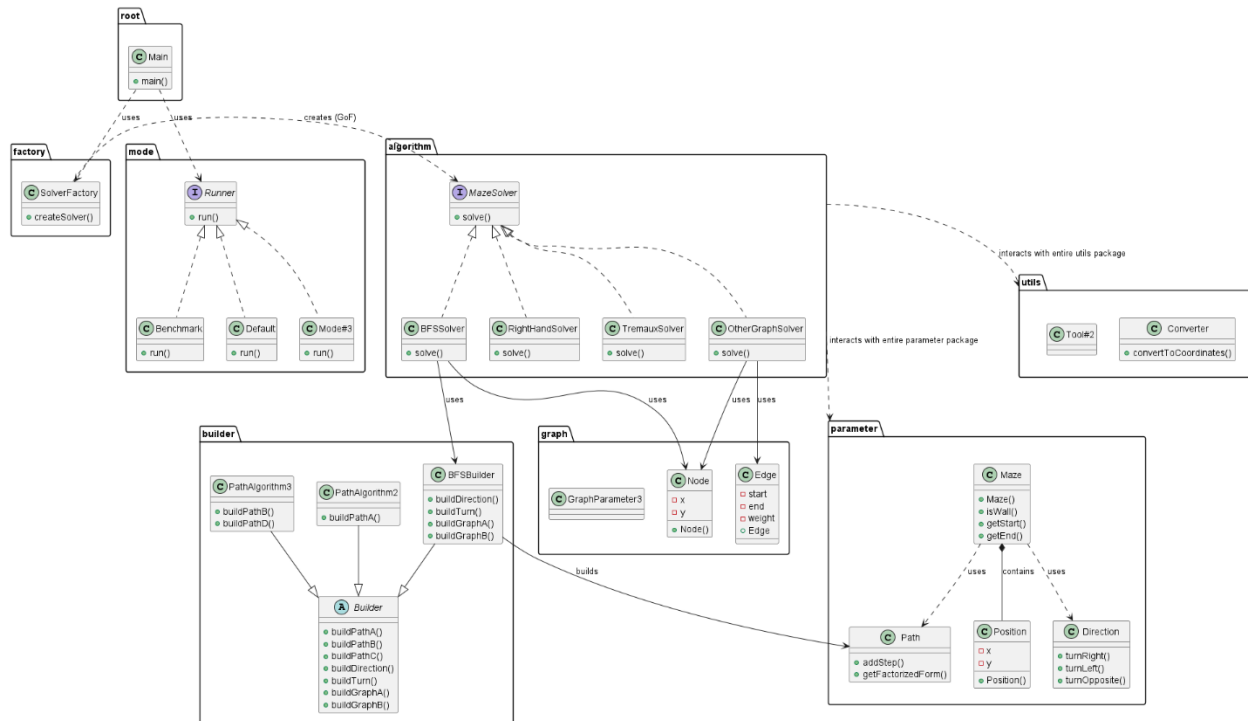
Start and End Nodes: Has been implemented in A1. Can be used in graph-based algorithm directly.

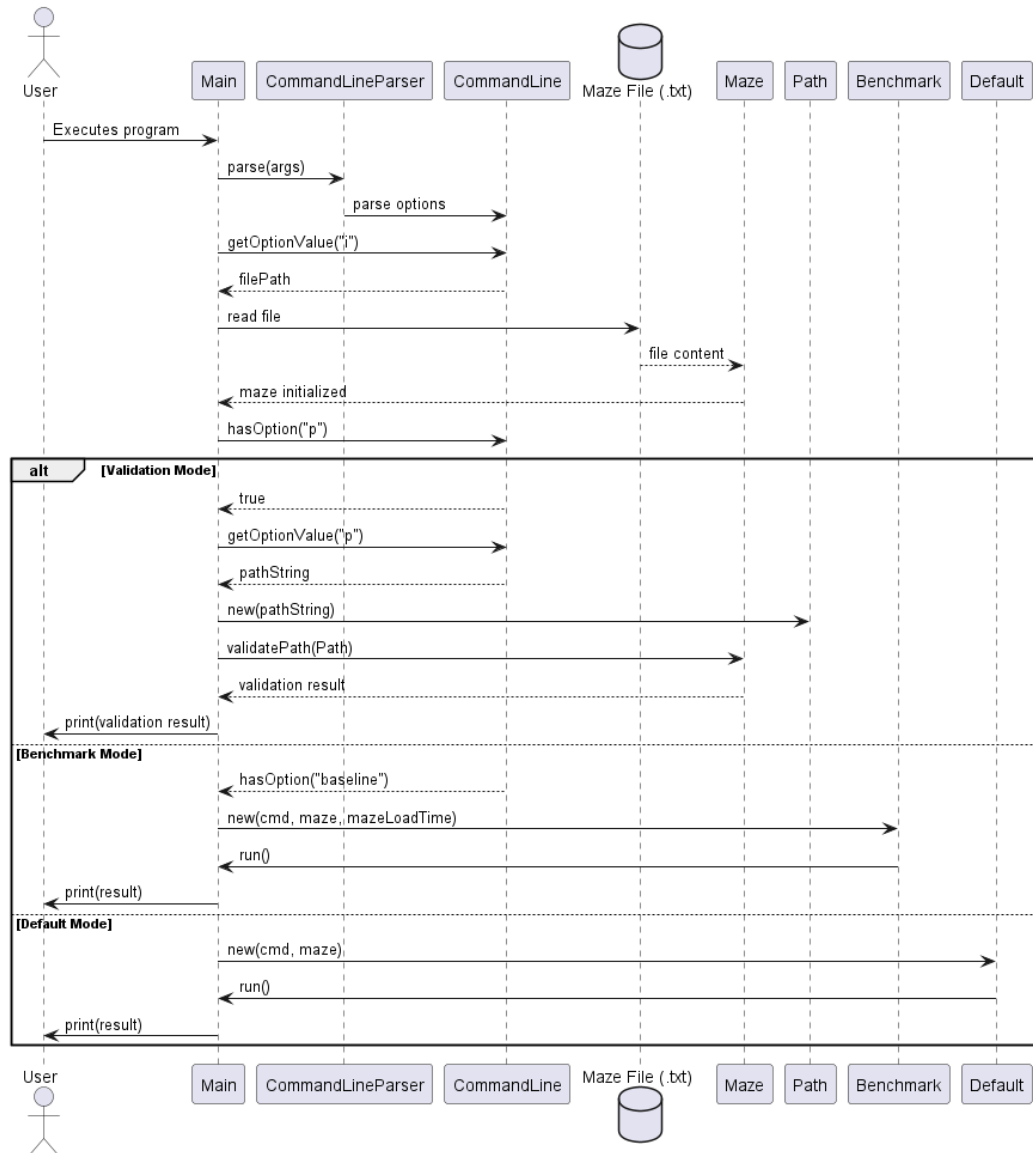
In general, unlike normal path-based parameters in A1 (e.g. position, direction), A3 graph-based algorithm will build on more abstract parameters.

Which kind of algorithms can be used to support such exploration on top of a graph ?

Breadth-First Search (BFSSolver) is the simplest one to implement since it can be implemented only based on nodes. In the future, we can also implement Depth-First Search (DFSSolver), and Dijkstra's Algorithm interface and its variant (DijkstraSP, DijkstraUndirectedSP, DijkstraAllPairsSP...)

STEP #3: FROM CRAPPY CODE TO DESIGNED ONE





STEP #4: CODE & TEST

How do you support the evolution of this Maze Runner in the future? Identify what can vary in this product and to what extent you are (or not) “protected from variations”.

To support the future implementation of the Maze Runner, the design should add new features and changes with minimal impact on the existing code (SOILD). Here is my decomposed solution:

Algorithm Package: New solving algorithms can be added without affecting the existing ones, thanks to the MazeSolver interface. This setup supports the Open/Closed Principle, allowing the addition of new solvers like BFSSolver, RightHandSolver, and TremauxSolver without modifying the interface or other solvers.

Builder Package: The Builder class allows for extending maze path-building capabilities, such as adding a new builder for a different path generation strategy.(e.g. path based, graph based, hybrid based...)

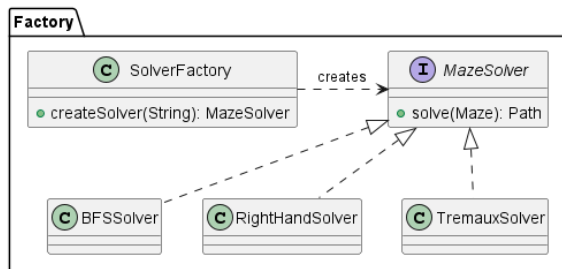
Mode Package: Different modes of operation, like Benchmark and Default, can be expanded or added to handle different runtime behaviors for clients' requirements, facilitated by the Runner interface.

Factory Package: The SolverFactory class abstracts the instantiation logic of maze solvers, adhering to the Factory Method pattern. This setup isolates the creation logic and supports adding new solvers easily.

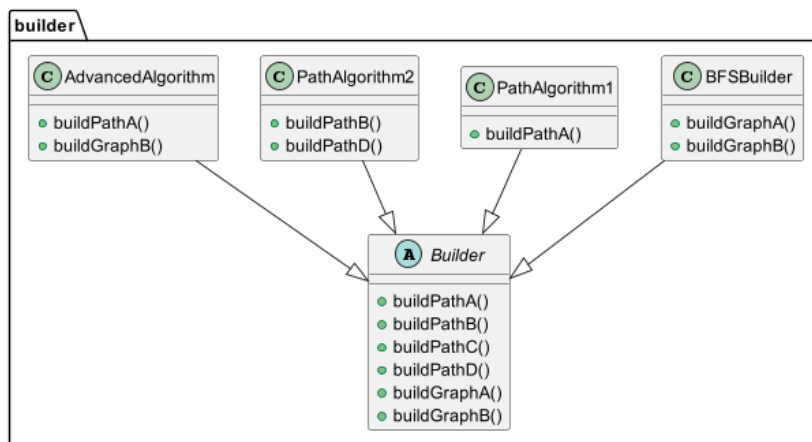
Utilization of Interfaces and Abstraction: Interfaces like MazeSolver and Runner reduce coupling between components, making the system more adaptable to changes.

If you have used some GoF or GRASP patterns: Which ones? Where? Why?

Factory pattern: SolverFactory in /Factory. I used this pattern aim to create instances of different types of MazeSolver without specifying the exact class of the object that will be created. This allows adding new solver algorithms without modifying the existing factory logic.



Builder pattern (Simple version, may need director for high-level algorithm and its variant): Builder and BFS
Builder in /builder. I used this pattern aim to separate the construction of a complex object (Path and Graph) from its representation, allowing the same construction process to create different representations. Because path-based and graph-based algorithms are different in primary output. We can prebuild the methods buildPathA(), buildPathB() etc. and buildGraphA, buildPathB() etc. to provide different implementations for different concrete builders, e.g. BFSBuilder, a graph-based algorithm, used buildGraphA and buildPathB(). This pattern allows me vary algorithm's internal representation and provides control over the steps of the construction process.



One must not test 100% of their product but focus on what is relevant. How did you identify what to test? To what extent do your tests give you confidence in your code?

To identify what to test in the project, I focused on several key aspects and converted them into actual Junit test files in /test.

Basic Functionality (**BFSSolverTest**, **PathTest**): The essential parts of the Maze Runner project are the pathfinding algorithms (BFSSolver, RightHandSolver, TremauxSolver). These are essential for the application's main purpose, which is to find a path through the maze. Therefore, testing these components to ensure they accurately find the path, handle “contradicted” cases, and operate efficiently on various maze sizes.

Maze Loading(**MazeTest**): The Maze class's ability to load and parse different maze files correctly is critical. Tests should verify that the maze is loaded correctly, the start and end points are identified correctly, and walls and open spaces are mapped accurately.

Error Handling(**SolverFactoryTest**): Ensuring the runner handles invalid inputs or unsolvable mazes is important. Tests should cover scenarios like invalid file paths, improperly formatted maze files, mazes without a valid start or end point, and mazes that are unsolvable.

```
-----
T E S T S
-----
Running ca.mcmaster.se2aa4.mazerunner.BFSSolverTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.829 s -- in ca.mcmaster.se2aa4.mazerunner.BFSSolverTest
Running ca.mcmaster.se2aa4.mazerunner.MazeTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.085 s -- in ca.mcmaster.se2aa4.mazerunner.MazeTest
Running ca.mcmaster.se2aa4.mazerunner.PathTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in ca.mcmaster.se2aa4.mazerunner.PathTest
Running ca.mcmaster.se2aa4.mazerunner.SolverFactoryTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.073 s -- in ca.mcmaster.se2aa4.mazerunner.SolverFactoryTest
```

STEP #5: BENCHMARK & COMPARE

What other kind of metrics could be helpful for this product? Any idea how to measure them?

Metrics	Measurement
Space complicity	Compare the memory usage of different algorithm implementations to determine efficiency improvements. Use big O notation for theoretical analysis (e.g., O(new) vs. O(old)) and monitor the application's memory release during execution for practical measurement. Note that we can assume the output based on 2C03 textbook and prove through actual coding. E.g. Breadth First Search is O(n) (n = number of nodes) in theory. While Algorithm BellmanFordSP is O(1/2n), so we can run the code to prove that BellmanFordSP use less space than BFS.
Time Complexity	Similar to Space complicity, Evaluate the algorithm's execution time in big O notation for theoretical analysis and measure actual run time during execution to compare performance.
Execution Stability	Execute the algorithm multiple times under the same conditions to check for consistency in performance and results. This is important to ensure that some hybrid algorithm (e.g. AcyclicSP, combined with path-based and graph-based) doesn't cause “deadlock” in complex/giant mazes.